

GreenEvo

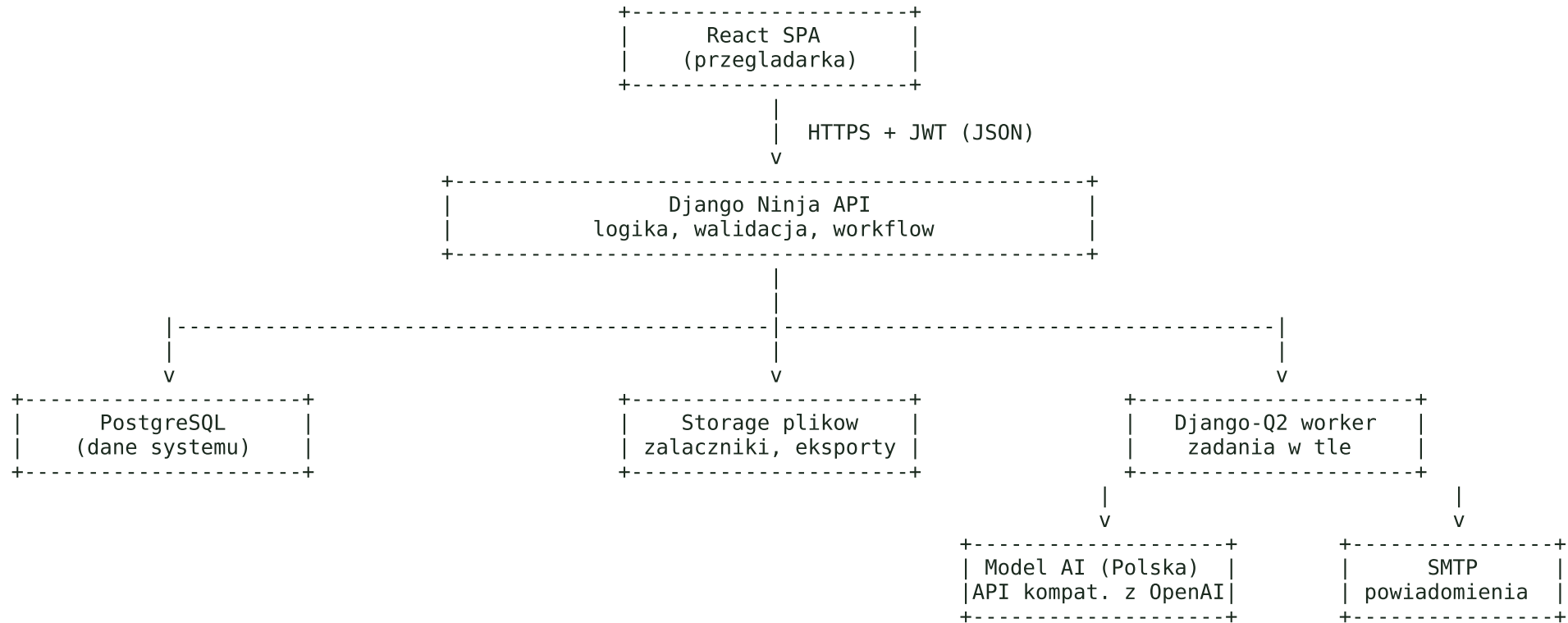
Architektura systemu backendowego

Projekt GREENSTRAT — MVP narzędzia do oceny wniosków w programie GreenEvo

Materiał roboczy na spotkanie — 7 września 2026

Django Ninja Extra - PostgreSQL - React (frontend) - Model AI hostowany w Polsce

Architektura komponentów



Podział na aplikacje i zależności

```
+-----+
| evaluation_model |
| matryca ocen jako dane |
+-----+
```

<--- korzysta z ---

```
+-----+
| ai_assist |
| integracja z modelem AI |
+-----+
```

--- korzysta z --->

```
+-----+
| applications |
| wniosek, workflow, eksperci, |
| raport z wizytacji |
+-----+
```

<--- korzysta z ---

```
+-----+
| accounts |
| użytkownicy, role |
+-----+
```

--- korzysta z --->

```
+-----+
| reporting |
| podsumowania, eksporty |
+-----+
```

Strzałki = kierunek zależności ("korzysta z"). *evaluation_model* i *accounts* nie zależą od nikogo — *reporting* korzysta ze wszystkich pozostałych.

Kluczowe encje wg aplikacji

accounts

User (email, role: APPLICANT | ADMINISTRATOR | EXPERT, is_active)

evaluation_model [dane wersjonowane, nie kod]

EvaluationModelVersion

-> Matrix (A / B / C, udział procentowy)

-> Criterion (A1..A8 / B1..B7 / C1..C9)

-> AnswerLevel (poziom 1-5, opis)

-> ScoreMultiplier (poziom -> przelicznik 0.05 .. 1.00)

-> FormalRule, DisqualifyingRule

applications

RecruitmentStatus (singleton: czy nabór aktywny)

Application (status, evaluation_model_version, submitted_snapshot)

-> ApplicantInfo (dane firmy, wpisywane przy każdym wniosku)

-> EcoInnovationInfo (opis technologii, obszary ESRS, SDGs)

-> CriterionAnswer (odpowiedź + uzasadnienie, per kryterium)

-> Attachment

-> ExpertReview (sugestia systemu vs. korekta eksperta)

-> VisitReport (raport z wizytacji)

-> StatusTransition (log audytowy przejść statusu)

ai_assist

AIAnalysisResult (append-only, powiązany z Application + Criterion)

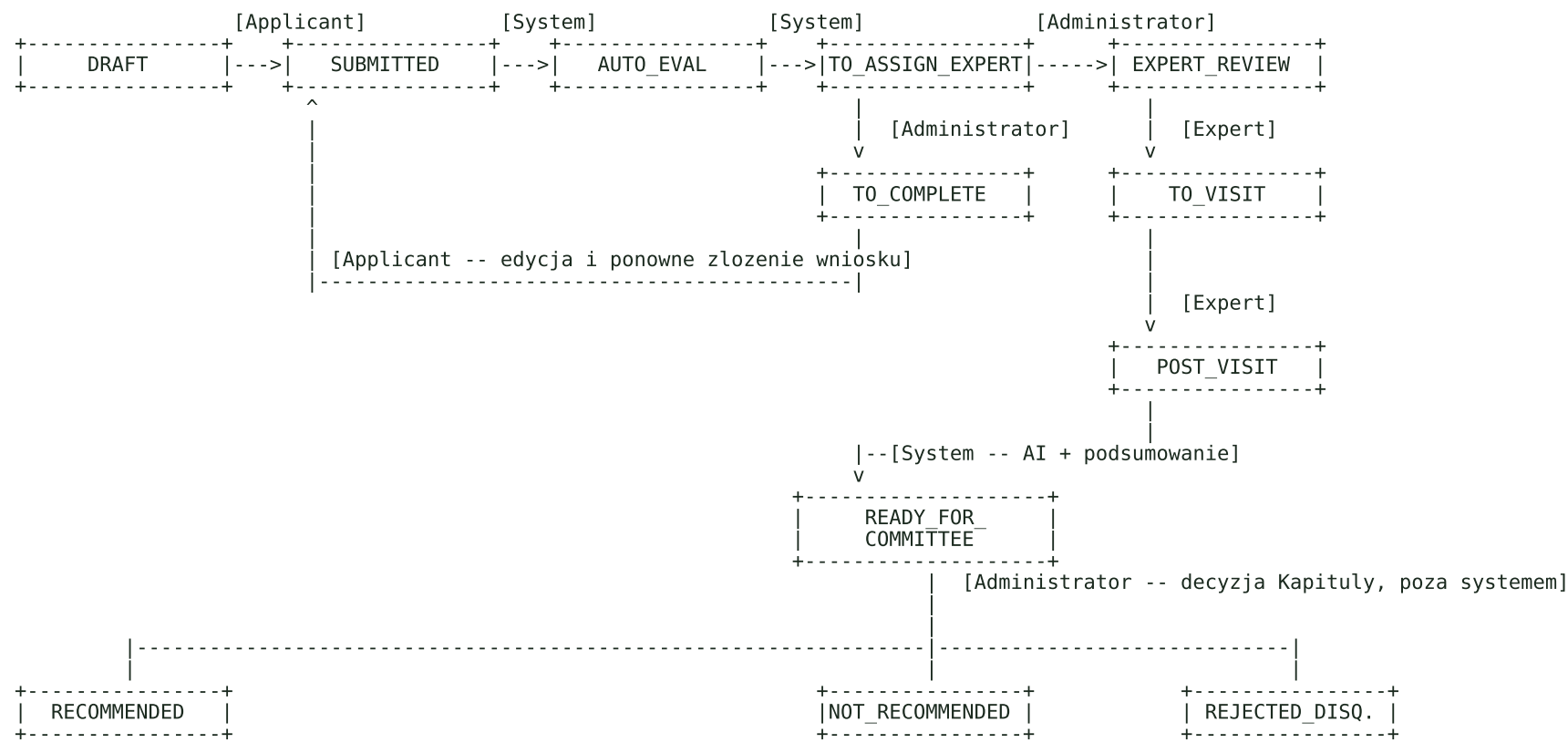
reporting

ApplicationSummary (generowany automatycznie po wizytacji)

BulkExport (zbiorcze zestawienie dla Kapituły, plik Excel)

NotificationLog

Maszyna stanow (12 statusow)



Korekty eksperta w *EXPERT_REVIEW* nie zmieniają statusu — dopiero zatwierdzenie wywołuje przejście.

Biblioteki i uzasadnienie wyboru (1/2)

■ Django

Dojrzały framework webowy w Pythonie z bogatym zestawem wbudowanych narzędzi (ORM, migracje, panel admina, mechanizmy bezpieczeństwa). Wybrany, bo to podstawa roli (Django backend developer) i bo minimalizuje ilość kodu do napisania samodzielnie przy napiętym terminie (3 miesiące na MVP).

■ Django Ninja Extra

Warstwa API REST na Django w stylu FastAPI — schematy danych to modele pydantic, kontrolery klasowe z dekoratorami `@route`, OpenAPI generowane automatycznie z type hintów. Wybrana zamiast DRF: jeden spójny mechanizm walidacji (pydantic) w całym projekcie, bo i tak jest potrzebny do walidacji odpowiedzi modelu AI, oraz automatyczna dokumentacja API bez ręcznych adnotacji na licznych custom-action endpointach (przejścia FSM).

■ PostgreSQL

Relacyjna baza danych — decyzja przyjęta wprost ("100%"). Dobrze pasuje do wymagań audytowalności (transakcyjność, integralność referencyjna) i obsługuje typy przydatne w projekcie (JSONField do przechowywania ustrukturyzowanych wyników AI i podsumowań).

■ django-ninja-jwt

Uwierzytelnianie JWT (access + refresh token) dla Django Ninja. Ten sam koncept co `django-rest-framework-simplejwt`, dobrany pod zmieniony framework API — backend i frontend są rozdzielonymi aplikacjami, JWT to standardowe, bezstanowe rozwiązanie dla takiej architektury, łatwe do obsługi po stronie SPA.

■ django-fsm-2

Aktywnie utrzymywany fork `django-fsm` — modelowanie maszyny stanów bezpośrednio na modelu Django przez dekorator `@transition`. Wybrana zamiast ręcznego słownika przejść: sama pilnuje, że nie da się wykonać niedozwolonego przejścia, a przejścia są czytelnymi, samodokumentującymi się metodami na modelu.

■ django-simple-history

Automatyczne wersjonowanie zmian na wskazanych modelach (osobna tabela historii per rekord, tworzona automatycznie). Wybrana, żeby zapewnić audytowalność zmian bez ręcznego pisania logiki zapisu historii przy każdej edycji pola.

Biblioteki i uzasadnienie wyboru (2/2)

■ **django-storages**

Abstrakcja nad przechowywaniem plików (lokalny dysk / S3-compatible / inne). Docelowy hosting nie jest jeszcze znany (zależy od Ministerstwa) — biblioteka pozwala zacząć na lokalnym dysku w dev i przełączyć się na storage w chmurze bez zmiany kodu, tylko konfiguracji.

■ **Django-Q2**

Lekki system kolejek zadań asynchronicznych działający na tej samej bazie Postgres, bez potrzeby osobnego brokera (jak Redis/RabbitMQ w Celery). Wybrany zamiast Celery, bo przy skali kilku wniosków miesięcznie osobny broker to zbędna infrastruktura — migracja do Celery jest możliwa później, gdyby skala istotnie wzrosła.

■ **openpyxl**

Generowanie plików Excel (.xlsx) w Pythonie. Wybrana, bo eksport zbiorczego zestawienia wniosków do Excela dla Kapituły Konkursu to wymaganie wprost ze specyfikacji systemu.

■ **django-environ**

Wczytywanie konfiguracji (sekrety, connection stringi) ze zmiennych środowiskowych / pliku .env w sposób typowany. Standard w projektach Django — pozwala rozdzielić ustawienia dev/test/produkcja bez trzymania sekretów w repozytorium kodu.

■ **pytest-django + factory_boy**

Bardziej zwięzły sposób pisania testów niż wbudowany unittest.TestCase (fixtures, parametryzacja) oraz generowanie danych testowych zamiast ręcznego tworzenia obiektów w każdym teście. Dodatkowa zależność, ale w praktyce przyspiesza pisanie testów.

■ **pydantic**

Walidacja i parsowanie danych przez modele z typowaniem. Używana do walidacji odpowiedzi JSON zwracanych przez model AI (LLM) oraz jako fundament schematów w Django Ninja Extra — jeden spójny mechanizm walidacji danych w całym projekcie, nie tylko w warstwie AI.

Zasada nadrzędna: każda dodatkowa zależność musi realnie oszczędzać czas lub rozwiązywać konkretne ryzyko — projekt ma bardzo napięty termin (3 miesiące na MVP).

Otwarte kwestie i następne kroki

Zasady nadrzędne przyjęte w architekturze:

- AI nigdy nie liczy punktacji — to deterministyczne obliczenie w Pythonie, AI ma rolę doradczą.
- Model oceny (kryteria, wagi) to dane, nie kod — gotowe na aktualizacje po badaniu ABCD Suzuki.
- Twarde rozdzielenie instrukcji systemowych od treści Wnioskodawcy w promptach AI (ochrona przed prompt injection).
- Każda decyzja/przejście statusu jest logowane — pełna audytowalność procesu.

Otwarte kwestie — czekają na Ministerstwo (nie blokują dalszej pracy nad kodem):

- Konkretny dostawca / model AI (PCSS czy inny polski dostawca chmury)
- Docelowe środowisko uruchomieniowe / hosting
- Wariant techniczny podpisu elektronicznego wniosku
- Okresy retencji danych
- Zasady finansowania / limitów kosztów AI